SplitMind: Towards Cost-Effective U-Shaped Split Learning for Private LLM Training

Abstract

Training large language models on private data is often infeasible for resource-limited local devices and risky for cloud platforms due to privacy concerns. U-shaped split learning (USL) offers a compelling solution by splitting the model into a U-shaped architecture, retaining both sensitive inputs of initial layers and outputs of final layers on the trusted client while transfering intermediate computations to the remote serve. Despite its promise, USL suffers from substantial GPU idle bubbles, high memory overhead, and a lack of automated parameter tuning, limiting its practical deployment.

To overcome these challenges, we present SplitMind, a streamlined USL training framework featuring three key innovations: (i) a U-shaped link-aware hybrid pipeline to minimize GPU idle time, (ii) bubble-bounded activation and model offloading to reduce GPU memory consumption, and (iii) an adaptive parameter configuration module for optimal setup selection. Extensive experiments demonstrate that SplitMind significantly outperforms standard and pipeline-optimized baselines (USL-FthenB and USL-1F1B). Notably, SplitMind successfully trains the Qwen3-32B model using only a local RTX 4090 GPU and three server-side A6000 GPUs, whereas baseline methods require six and five server A6000 GPUs, respectively. Furthermore, compared to USL-FthenB and USL-1F1B, SplitMind achieves training speedups of 1.58 and 1.89 times, alongside GPU rental cost reductions of 36.59% and 63.64%. Our code will be publicly available.

1 Introduction

Large language models (LLMs) have attracted widespread attention, increasing demand for customized services through fine-tuning on private, domain-specific data [5, 11, 64]. However, the reliance on powerful GPU clusters makes this process costly and often unaffordable for small and medium-sized enterprises and research teams [27, 31]. While cloud platforms offer a potential solution by providing scalable resources, concerns about data privacy often hinder organizations from sharing sensitive data with third-party servers [63]. Consequently, organizations face a fundamental dilemma: how to secure sufficient computational resources without compromising stringent data privacy.

To navigate this conflict between computational demands and privacy constraints, U-shaped split learning (USL) [35, 42, 47] offers a compelling solution by distributing model computation across clients and servers. Specifically, USL splits the model into three segments: *head* (initial layers), *trunk* (central layers), and *tail* (final layers). Under this paradigm, the trusted client retains control over the head and tail, while a powerful server manages the computationally intensive trunk. Data flows in a characteristic U-shaped pattern: from the client's head to the server's trunk and back to the client's tail during the forward pass, and inversely during the backward pass. This architectural separation ensures that raw data and output labels never leave the trusted client, thereby eliminating the risk of direct dataset exposure.

While protecting raw data, the intermediate data transmitted in USL remain vulnerable to data reconstruction attacks (DRAs) [16, 46]. Malicious actors can invert smashed data [7, 19, 24] or match gradients [3, 10, 72] to deduce original inputs and labels. To counter this threat, perturbation-based defense methods inject differential privacy noise [15] to intermediate activations [14, 37, 61] or gradients [1, 49, 62] to thwart reconstruction attempts. Recent advances DualGuard [34] further fortify USL via a parameter space transformation approach to achieve bidirectional protection. Consequently, USL, augmented with these robust defenses, emerges as a practical and secure framework for analyzing sensitive data with limited local hardware resources, making it highly suitable for deployment in regulated domains.

Despite a growing body of work on algorithmic attack and defense optimizations in USL, existing open-source implementations rely on single-GPU emulation that sidesteps real network communication [7, 34], executing all model segments on a same local GPU and extracting intermediate activations directly via memory access. This abstraction fails to account for realistic network latency, bandwidth, and hardware disparities, leaving critical systems challenges underexplored. To bridge this gap, we developed a standard USL training process in a realistic system environment, revealing significant systemic issues:

Substantial GPU idle bubbles. The U-shaped dataflow in USL causes substantial GPU idle bubbles due to frequent synchronization and transfer overheads. While pipeline parallelism techniques like F-then-B [28] and 1F1B [17, 39–41, 45, 48] can overlap computation and communication, they are less effective in USL scenarios with U-shaped training dependency and constrained bandwidth for client-server communication. Moreover, shared hardware between the client-side head and tail stages restricts true parallel overlap, further diminishing efficiency. Consequently, even with pipeline optimizations, client GPU idle rates remain high (e.g., 84.73%and 79.12%for F-then-B and 1F1B, respectively).

High GPU memory comsumption. USL creates a significant memory bottleneck on the client, which must retain critical components like the embedding layer, first block, final block, and LM Head. This issue is exacerbated by LLMs

storing massive parameters, activations, and optimizer states, rapidly exhausting the limited GPU memory available on client. Inefficient resource allocation further worsens the problem: when the head is active, the idle tail remains memory-resident, and vice versa. This persistent underutilization of client GPU memory forces most of the computational load onto rented remote servers, substantially increasing financial costs in commercial deployments.

Lack of automated parameter tuning. The efficiency and cost of USL training are critically dependent on configuration parameters, such as split points and batch sizes. Current methods rely on either costly trial-and-error [21, 36] or error-prone symbolic formula estimation [35, 68]. Although profile-and-simulate techniques have been successfully applied to pipeline-parallel LLM training [17, 39, 48, 56, 67], they are inadequate for USL. They cannot model the complex interplay of client-resident head and tail stages, U-shaped execution dependencies, and cross-device hardware heterogeneity. This gap leaves a critical need for automated parameter tuning tailored specifically to the USL framework.

To tackle these challenges and facilitate USL deployment, we present SplitMind, a streamlined USL training framework that co-optimizes pipeline scheduling, memory management, and automated parameter tuning. By integrating these components, SplitMind bridges the gap between algorithmic USL research and its practical, large-scale deployment, enabling efficient fine-tuning of LLMs on resource-constrained hardware. Our key contributions include:

- We establish a standard USL training process in a realistic system environment and adapt classic pipeline parallelism methods (i.e., F-then-B and 1F1B) to the USL context. Through comprehensive experiments, we identify and quantify three key efficiency bottlenecks hindering the deployment of current USLs.
- We present a novel *U-shaped link-aware hybrid pipeline* tailored for USL, which synergistically combines F-then-B scheduling on head and trunk stages, and 1F1B scheduling on tail stage, effectively eliminating unnecessary waiting times and reducing client idle periods.
- We design a *bubble-bounded inactive-tensor offloading*, which optimizes the offloading of model states and activations by exploiting idle bubbles in the USL dependency graph. It alleviates GPU memory constraints while ensuring data transfer time is overlapped with computation and communication.
- We introduce an *adaptive parameter configuration* that automatically determines the optimal model split point and batch size based on varying client resource constraints, server costs, networks, aiming at minimizing server rental costs and maximizing training efficiency.
- We implement SplitMind and conduct extensive experiments using Qwen3, LLaMA3 and Gemma2 models, ranging from 4B to 32B in parameters. Results demonstrate that SplitMind consistently outperforms standard USL, USL-FthenB, and USL-1F1B. Specifically, SplitMind supports larger models and batch sizes while achieving notable enhancements in training time, server memory usage, and server rental cost, with reductions of 2.78%-76.07%, 0.25%-90.26%, and 22.60%-89.64%, respectively, compared to baselines.

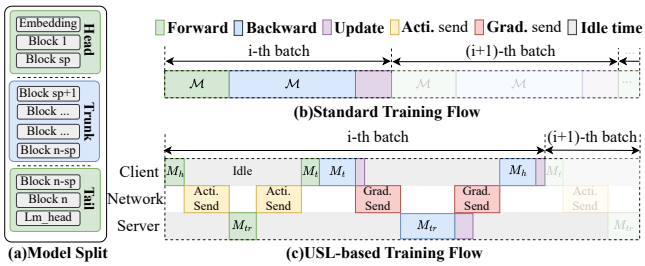


Figure 1. Illustration of (a) model split, (b) standard model training flow and (c) USL-based model training flow.

2 Background and Motivation

2.1 USL-based LLM Training

Split learning (SL). The need to fine-tune LLMs using private data is hindered by limited computational resources on local devices and data privacy concerns when using cloud platforms. To bridge the gap, split learning (SL) [21, 60, 65] has emerged as an innovative architecture. SL splits the model into a local client part for processing raw data and a remote server part for handling main training, exchanging only intermediate activations and gradients to reduce local computational load while protecting raw data. However, the auto-regressive nature of LLMs can lead to overlapping input data and output labels, potentially allowing the server to infer the original data from the output labels [7, 34].

U-shaped split learning (USL). USL [35, 42, 47] improves upon standard SL by splitting the model into three segments: *head* (initial layers), *trunk* (central layers), and *tail* (final layers), as shown in Figure 1(a). Local client oversees the head and tail, while remote server handles the trunk. The training processes of a standard LLM and a USL-based LLM are depicted in Figure 1(b) and Figure 1(c), respectively. In the USL training process, data flows in a U-shaped pattern. During the forward pass, client's head model processes input data, sends intermediate activations to server. Server then processes these activations through trunk model and returns them to client's tail model for loss computation. The backward pass follows a similar flow: gradients are computed by client's tail model, sent to the server for backpropagation, and then returned to update the head model.

Privacy protection in USL. To further prevent attacker use the transmitted intermediate data to deduce original data, A various defense mechanisms are proposed to fortify the USL architecture. Perturbation-based techniques, especially

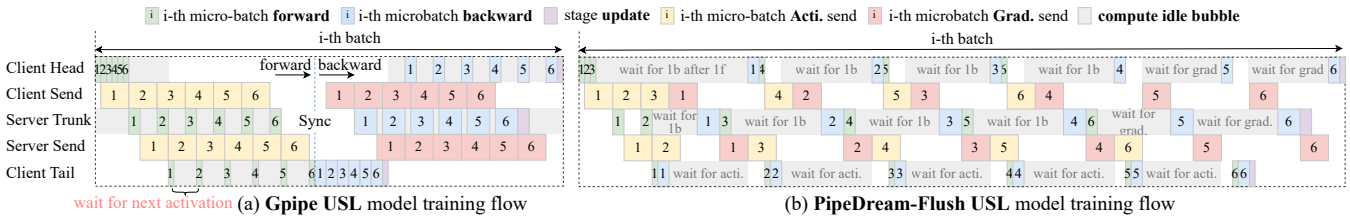


Figure 2. Illustration of (a) F-then-B, (b) PipeDream for USL training with 6 micro-batches. The red box indicates the training flow of the first micro-batch. Note that the client head and tail stages are actually located on the same node.

leveraging differential privacy (DP) [15], are commonly used to modify sensitive attributes and enhance privacy protection. For forward smashed data inversion attacks, DP noise is added to either the original data output [25] or intermediate activations [13, 14] to prevent reconstructing the original data. To address backward gradient matching attacks, introducing noise into gradients can help thwart attempts to reconstruct label information [1]. Besides, DualGuard [34] further propose a bidirectional defense mechanism tailored for split-based LLM-FT scenarios via a parameter space transformation approach. These state-of-the-art solutions effectively establish USL as a privacy-preserving architecture that guarantees data confidentiality without compromising downstream task performance. This makes USL highly suitable for practical, highly-regulated domains, such as analyzing sensitive medical or financial data.

Computational inefficiencies in USL training flow. While existing USL research has primarily focused on algorithmic-level attack and defense optimizations, their open-sourced implementations rely on single-GPU emulation that sidesteps real network communication [7, 34]. To address this void, we establish a standard USL training process in a realistic system environment, uncovering significant computational inefficiencies arising from costly client-server synchronization and communication. Specifically, the U-shaped dataflow requires four communication rounds per training iteration: two for forward activations and two for backward gradients. In typical client-server wide-area network (WAN) environments, low bandwidth and high latency severely degrade training performance, leading to substantial GPU idle bubbles, as visualized in Figure 1(c).

2.2 Pipeline Optimized USL Training

Given USL's model-parallel nature, we further explore advanced pipeline parallelism techniques, such as F-then-B [28] and 1F1B[39], which break training batches into smaller micro-batches that are processed simultaneously across stages, enabling overlapping computation and communication to enhance training efficiency. By customizing these techniques, we can enhance USL training scheduling.

F-then-B optimized USL (USL-FthenB). *F-then-B pipeline scheduling*, introduced by GPipe [28], divides a training batch into multiple micro-batches. It first completes the forward pass for *all* micro-batches, followed strictly by the backward pass in reverse order, as depicted in Figure 2(a). Gradients are accumulated across all micro-batches, and model updates are synchronized only after the entire batch is processed. Although this interleaving of micro-batches across stages effectively reduces GPU idle bubbles, it necessitates caching all intermediate activations throughout the forward pass, leading to significant memory pressure.

1F1B optimized USL (USL-1F1B). The *1F1B pipeline scheduling* addresses the memory challenges of F-then-B through a refined interleaved schedule. After a warm-up phase, each pipeline stage alternates between forward and backward passes for different micro-batches, as illustrated in Figure 2(b). This approach reduces peak memory usage by releasing activations promptly after their corresponding backward pass. However, its asynchronous execution can cause weight inconsistencies across stages and hinder convergence. While PipeDream [39] and PipeDream-2BW [40] manage weight consistency using multiple versions or double-buffered weights, these methods increase GPU memory consumption. PipeDream-Flush [40] adopts a synchronous 1F1B variant to ensure parameter consistency, albeit with additional synchronization overhead. Megatron-LM [41] reduces bubbles through interleaved pipeline scheduling, and ZeroBubble [48] further fills idle periods by decomposing backward propagation into input-gradient and weight-gradient computations. Although these optimizations improve GPU utilization in tightly-coupled GPU clusters, they fail to address core USL bottlenecks, such as shared client-side execution and WAN communication constraints.

2.3 Efficiency Issues in Practical Deployment

Despite adopting advanced pipeline optimizations, the practical deployment of USL training still faces significant efficiency challenges that impede its real-world application.

Limitation #1: significant GPU idle bubbles and inefficient pipeline. The U-shaped dataflow of USL introduces significant GPU idle bubbles. While current pipeline parallelism techniques like F-then-B and 1F1B are designed to overlap computation and communication in high-bandwidth, low-latency clusters with fast interconnects (e.g., NVLink or InfiniBand) [43], they prove less effective in typical USL environments characterized by limited bandwidth (often below

Table 1. Training costs with different USL frameworks.

	Naive USL	USL-FthenB	USL-1F1B	SplitMind
Epoch training time (h)	115.68	44.32	67.04	42.24
Client memory (GB)	19.57	19.40	22.06	19.27
Client idle	96.13%	84.73%	79.12%	59.26%
Server memory (GB)	64.52	59.60	20.93	18.86
Server GPU Num	2	2	1	1

300 Mbps) and high latency [20]. As shown in Figure 2, in F-then-B scheduling, the synchronization between forward pass and backpropagation causes idle bubbles during the tail stage's forward pass due to network delays in receiving the next micro-batch. 1F1B scheduling enforces a strict alternation between forward and backward passes, leading to idle waiting periods after the forward pass when the subsequent backward pass is waiting for transmitted gradients. Furthermore, the head and tail stages, though conceptually separate, often share the same client hardware, restricting true parallel overlap and further impacting pipeline efficiency.

To validate these inefficiencies, we implemented Naive-USL, USL-FthenB, and USL-1F1B in a practical system setup comprising a client with an RTX 4090 (24GB memory) and a server with 8×A6000 GPUs (48GB memory each). Using the Qwen3-8B model on the UltraChat-200k dataset with a sequence length of 512 and enforcing a network bandwidth constraint of 230 Mbps [20], we evaluated the methods under their optimal configurations (determined via Figure 8). As presented in Table 1, Naive-USL exhibits a severe client idle ratio of 96.13%, requiring 115.68 hours per epoch and 64.52 GB of server memory (need two A6000 GPUs). While USL-FthenB reduces training time to 44.32 hours, it maintains high memory usage. Conversely, USL-1F1B lowers server memory to 20.93 GB at the expense of a higher time cost (67.04 hours). Despite these improvements, both methods still suffer from client idle ratios exceeding 79%. In contrast, our SplitMind significantly enhances training efficiency, reducing the time to 42.24 hours, server memory to 18.86 GB (only need one A6000 GPU), and the client idle ratio to 59.26%.

Limitation #2: high GPU memory comsumption on the client. The USL framework imposes a significant memory bottleneck on the client due to three primary constraints. First, critical components, like the initial embedding layer, first block, final block, and the ending LM Head, must reside on the client for architectural and safety reasons, creating a static GPU memory demand. For instance, fine-tuning Qwen3-8B model with a sequence length of 512 and a batch size of 16 requires 16.36 GB of client GPU memory for this static demand, which increases with larger batches. Second, LLMs store massive parameters, intermediate activations, and optimizer states within each model layer, rapidly exhausting the client's limited GPU memory even with only a few layers. Last, memory allocation is inherently inefficient: when head segment is active, memory allocated to inactive tail segment remains occupied, and vice versa, leading to persistent underutilization of constrained GPU memory.

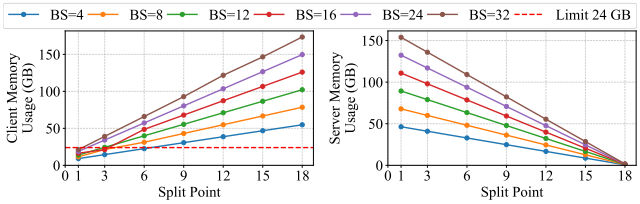


Figure 3. Peak GPU memory usage on different settings.

To quantify this limitation, we measure peak GPU memory consumption during fine-tuning of a 36-layer Qwen3-8B model using USL-FthenB. Experiments summarized in Figure 3 varied the batch size and the model split point (*SP*), where the head, trunk, and tail segments contain *SP*, $36-2\cdot SP$, and *SP* layers, respectively. Results reveal a critical trade-off: increasing *SP* allocates more layers to the client's head and tail, raising peak GPU memory consumption on the client while reducing the server-side memory demand. However, this trade-off becomes critically limiting under constrained client hardware, such as a single NVIDIA RTX 4090 GPU (24 GB memory). For batch sizes of 4, 8, 16, and 32, the maximum feasible *SP* is constrained to 6, 4, 2, and 1 layer(s), respectively. Larger batch sizes trigger out-of-memory (OOM) errors even with a single layer split point, making USL training unfeasible. This constraint is more pronounced for larger models, shifting most of the computational workload to the trunk segment on rented cloud servers, significantly increasing the financial costs of commercial deployments.

Limitation #3: lack of automated parameter tuning. The effectiveness of USL training depends heavily on key hyperparameters, like split point and batch size, which determine the distribution of computational workload between the client and server, impacting resource usage and training efficiency. Existing approaches often rely on empirical trial-and-error [6, 21, 36], which manually enumerates a small set of candidate configurations, runs pilot executions for each, and select the best performer. This process is costly in both time and hardward demand, relies on user experience, and often misses optimal settings in a large parameter space.

Recent studies have explored *symbolic derivation* to formulate and estimate training time and memory demand based on theoretical hardware parameters (e.g., nominal bandwidth, peak throughput) without full training runs [35, 68]. They typically define computation time as $T_{comp} = \frac{B\cdot\omega\cdot\kappa}{f}$ and communication time $T_{comm} = \frac{D_{trans}}{R}$, where B is the batch size, ω is the computational workload per sample (in FLOPs), κ is the hardware cycles needed per workload unit, and f denotes the device's computing frequency, D_{trans} is the data transmission volume, and R denotes the uplink transmission

Table 2. Performance of different configuration search methods, i.e., empirical search (Emp.), symbolic derivation (Symb.), and our simulator-based approach. Values in parentheses denote the relative error compared to the actual test results.

Method	Search time	Step time (second)	Server Mem. (GB)	Client Mem. (GB)
Standard USL				
Emp.	≈2 hours	7.03	18.75	17.28
Symb.	<1 second	6.29 **(10.5%)**	11.75 **(37.3%)**	9.49 **(45.1%)**
SplitMind				
Emp.	≈1.5 hours	2.97	8.53	13.93
Symb.	Inapplicable due to incorporated system optimizations			
Ours	< 2 minutes	2.95 **(0.7%)**	8.53 **(0.0%)**	13.81 **(0.9%)**

rate. Despite its speed, symbolic derivation struggles to accurately predict per-step execution times and total memory usage across different training phases, leading to significant estimation errors. Moreover, incorporating advanced system optimizations like pipeline parallelism or memory offloading renders direct symbolic derivation extremely challenging.

We experimentally validate these limitations using the Llama-3.2-1B model (batch size 8, sequence length 512), with results summarized in Table 2. In a standard USL setting, an empirical search for the optimal configuration via actual execution requires approximately 2 hours. In contrast, the symbolic derivation provides rapid estimates but introduces substantial errors: an average 10.5% error in per-step time and significant peak memory errors of 37.3% on the server and 45.1% on the client. In our proposed SplitMind setting, where symbolic derivation is inapplicable, empirical search would take 1.5 hours. However, our simulator-based estimation completes in just 2 minutes, achieving a markedly lower estimation error of only 0.7% for step time, with memory-usage errors of 0.0% on the server and 0.9% on the client. This demonstrate that our approach offers highly accurate performance predictions, facilitating rapid hyperparameter configuration for optimized USL environments.

3 Design of SplitMind

This section presents SplitMind, a novel framework tailored for USL, where a local client collaborates with a powerful server over a bandwidth-constrained network. Its core objective is to maximize training efficiency while minimizing the reliance on expensive server resources. To achieve this, SplitMind integrates three key innovations, as illustrated in Figure 4: (1) a *U-shaped link-aware hybrid pipeline* that optimizes the overlap between computation and communication to reduce idle time, (2) a *bubble-bounded inactive-tensor offloading* to alleviate GPU memory presure without compromising training speed, and (3) an *adaptive parameter configuration* to automatically determine the optimal model split point, batch size under different resource constraints.

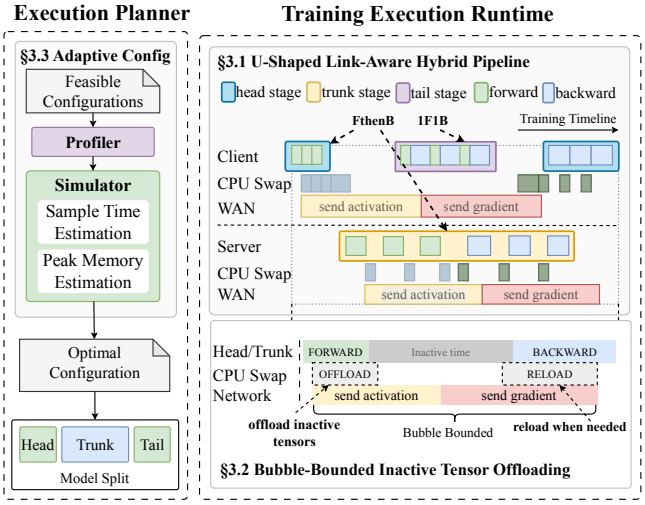
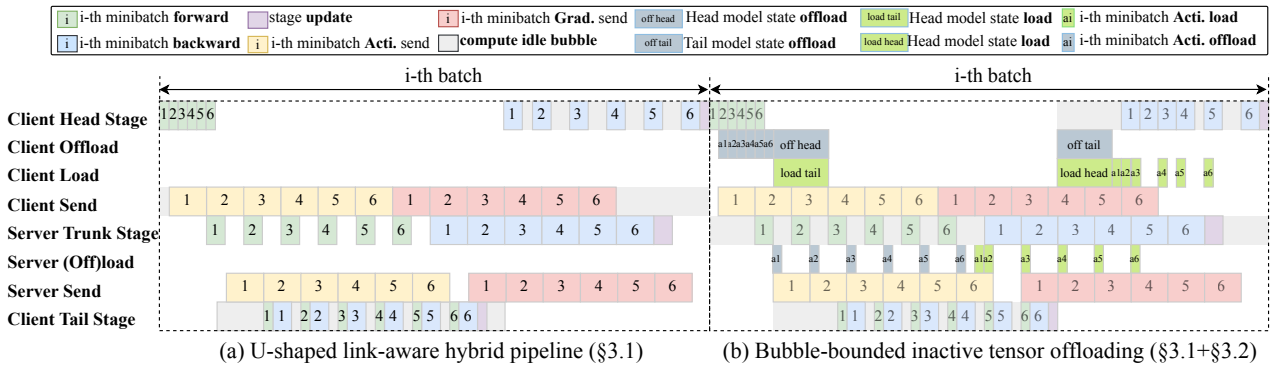


Figure 4. Overview of SplitMind framework.

3.1 U-Shaped Link-Aware Hybrid Pipeline

To address the significant idle bubbles inherent to existing USL frameworks, we propose a *U-shaped link-aware hybrid pipeline strategy*. This approach is specifically designed to optimize the overlap between computation and communication while adapting to the low-bandwidth network deployments, thereby reducing resource idle time and memory overhead. Our hybrid design combines the advantages of two established pipeline parallelism paradigms: the F-then-B strategy, which incurs useless waiting for next activation in client tail stage and high memory usage by caching activations for all micro-batches, and the 1F1B strategy, which forces the client to stall whenever the return of gradients is delayed by the network, thereby exacerbating pipeline bubbles. By strategically combining these approaches, we aim to mitigate their individual limitations for the USL context. As illustrated in Figure 5 (a), the core architecture decomposes the model into a three-stage pipeline. The client device manages both the head (initial layers) and tail (final layers) stages, while the server hosts the computationally intensive trunk stage (the central bulk of the model). During training, a large global-batch is partitioned into smaller micro-batches that are processed sequentially through this pipeline. A key innovation of our strategy is the employment of distinct scheduling policies tailored to the asymmetric temporal characteristics of each segment: we *adopt 1F1B at the tail stage* to mask data availability gaps caused by communication delay, while *deploying F-then-B at the head and trunk stages* to buffer computation against the prolonged gradient return delays inherent to U-shaped topology.

F-then-B scheduling at head and trunk. We employ a synchronous F-then-B scheduling approach for the head and trunk stages. In this process, all micro-batches complete their forward passes through the head and trunk stages before



(a) U-shaped link-aware hybrid pipeline (§3.1) (b) Bubble-bounded inactive tensor offloading (§3.1+§3.2)

Figure 5. Illustration of SplitMind training pipeline: (a) without offloading and (b) with offloading.

any backward propagation begins. Following the completion of backpropagation for all micro-batches of the head stage within the current batch, we conduct a stage update to adjust the model weights. This design choice prioritizes weight update consistency and simplifies scheduling logic. More importantly, it avoids the substantial idle time that can arise in strict 1F1B schedules under high network delay, where a node must wait for the backward pass of a micro-batch to finish before starting the forward pass of the next, creating a rigid coupling that exacerbates communication delays.

Alternatively, the trunk stage could adopt a mixed execution strategy where forward and backward passes are interleaved strictly based on their arrival order (i.e., FCFS policy). However, since backward may consume computation and communication resources required by forward, the forward execution of certain micro-batches can be delayed. As a result, the overall training process is still prolonged, as the later micro-batches finish much later, which does not effectively reduce the total training overhead. Therefore, we adopt a simpler and more predictable scheduling method, executing all forward passes before initiating backward passes. This approach simplifies synchronization between stages and maintains stable throughput under high-latency conditions.

1F1B scheduling at tail. We adopt an 1F1B scheduling strategy for the tail stage. Here, the processing of micro-batches in the tail stage is interleaved, with each micro-batch undergoing a forward pass followed immediately by its backward pass. This allows the tail stage to fully utilize the *waiting bubbles* that occur while activations are being transmitted from the server-side trunk model. Furthermore, this interleaved execution enables the immediate release of activations after their corresponding backward pass is finished, significantly reducing the peak memory consumption on the client compared to the F-then-B strategy, which retains all activations until the entire backward pass is completed.

This hybrid pipeline method efficiently mitigates GPU idle bubbles, enhances GPU utilization through increased concurrency, and offers a better trade-off compared to traditional monolithic pipeline strategies. Besides, under our hybrid pipeline, we have managed to avoid the shared hardware conflict problem that arises when both the head and tail stages compete for computing resources simultaneously.

3.2 Bubble-Bounded Inactive-Tensor Offloading

Inactive-tensor analysis in USL dataflow. The U-shaped dataflow of USL induces significant temporary inactivity among various tensors during training. Specifically, *forward activations generated by the trunk segments* remain dormant until the client completes the tail backward pass. Likewise, *forward activations from the head segments* are inactive pending the completion of both the client's tail and server's trunk backpropagation. Furthermore, the concurrent residency of *both head and tail model states* in client memory constitutes a major source of memory inefficiency.

USL-aware inactive-tensor offloading. While memory offloading is a natural remedy, existing methods typically frame it as a generic tensor-liveness or memory-capacity optimization within a monolithic training graph [9, 18, 50, 52]. This perspective fails to exploit a key characteristic of USL: the U-shaped pipeline creates deterministic, structured inactivity windows bounded by computation and communication bubbles. We therefore propose *bubble-bounded inactive-tensor offloading*, which strategically places offload and reload operations within these idle intervals across the head, trunk, and tail stages, managing inactive data based on their temporal redundancy. As shown in Figure 5(b), we apply distinct strategies tailored to different tensor patterns. For client-head and server-trunk activations, which are inactive post-forward and reused later during backpropagation, this window spans computation and WAN delays, making it ideal for per-micro-batch offloading. Since earlier micro-batches possess longer idle windows, they are prioritized. Client model states exhibit a complementary pattern: head states are active during head phases, and tail states during tail phases. We therefore swap these states at stage boundaries. In contrast, tail activations and server model states are mostly

active, rendering them unsuitable for offloading. These insights drive our strategy to target only client-head/server-trunk activations and client-head/tail model states.

Client-head and server-trunk activation offloading. Client-head and server-trunk activations share a key property: they are inactive post-forward until their corresponding backward pass begins. However, their inactive windows differ in the U-shaped pipeline. Head activations are generated early but wait until data traverses the trunk/tail and gradients propagate back. Trunk activations wait until the client processes them and returns gradients. Since these waits include WAN communication and downstream computation, they are long enough to hide offloading latency, creating natural opportunities for activation offloading. Our mechanism exploits this by asynchronously offloading these activations to CPU DRAM post-forward and reloading them pre-backward. Earlier micro-batches have longer inactive windows, so they are prioritized when only partial offloading is feasible. Trunk activation prefetching starts after the last micro-batch's forward pass. During each micro-batch's backward computation, the next micro-batch's activations are fetched from CPU DRAM. Head activations follow the same principle and are reloaded before the head backward phase. This design overlaps data movement with computation/communication, reducing GPU memory pressure without stalling execution.

Client-side head and tail model-state offloading. Storing both head and tail model states on the client GPU creates significant memory pressure. However, the U-shaped pipeline makes these stages mutually exclusive: the head is active only during initial forward and final backward phases, while the tail is active only after the trunk returns activations. We exploit this via stage-level model-state swapping. Initially, head states are on the GPU and tail states are in CPU DRAM. After all micro-batches finish the head forward pass, head states are offloaded and tail states are loaded for tail forward/backward computation. Once the tail finishes, its states are offloaded and head states are reloaded for head backward propagation. This policy keeps only the active head or tail stage on GPU and overlaps swapping with communication or computation, drastically reducing GPU memory cost.

Crucially, the time overhead of offload and reload operations is fully masked by concurrent operations, minimizing performance penalties and achieving efficient memory utilization without compromising throughput. We experimentally validate the impact of our offloading mechanism on memory usage and training time in §4.4.

GPU memory cost analysis. We analyze the peak GPU memory consumption of different pipeline strategies. Let n denote the number of micro-batches per global batch, and d denote the pipeline depth (warm-up steps, where $3 \leq d \ll n$). We define the following memory metrics: Let W_c denote the total model states (including parameters, optimizer states, and gradient buffers) on the client. Let A_h and A_t denote the activation memory per micro-batch for the head and tail stages, respectively. W_s and A_s represent the corresponding metrics for the server trunk. Table 3 summarizes the maximum memory footprint. On the client side, beyond the fixed model states W_c, the memory consumption of Naive-USL and USL-FthenB is dominated by the linear accumulation of activations ($n \cdot A$), as they must retain tensors for the entire batch. While our U-shaped link-aware hybrid pipeline (ULHP) reduces the tail activation memory to A_t, its peak memory remains bottlenecked by the linear activation dependency of the head stage ($n \cdot A_h$). In contrast, USL-1F1B mitigates this issue by scaling activation memory with the pipeline depth d rather than the batch size n. SplitMind (with ULHP and bubble-bounded inactive-tensor offloading) achieves the lowest client-side memory footprint by combining stage swapping and activation offloading. Specifically, stage swapping reduces the resident model states to approximately $0.5W_c$, while activation offloading decouples activation memory from n, yielding near-constant peak memory consumption ($O(1)$). On the server side, beyond the fixed trunk model states W_s, Naive, USL-FthenB, and our ULHP exhibit activation memory that scales linearly with n, whereas USL-1F1B scales with the pipeline depth d. SplitMind removes these dependencies by offloading inactive tensors, maintaining a constant activation memory cost ($O(1)$) independent of the batch size and thereby significantly improving scalability.

Table 3. Comparison of peak memory usages analysis.

Method	Client Peak Memory	Server Peak Memory
Naive-USL	$W_c + \mathbf{n} \cdot A_h + \mathbf{n} \cdot A_t$	$W_s + \mathbf{n} \cdot A_s$
USL-FthenB	$W_c + \mathbf{n} \cdot A_h + \mathbf{n} \cdot A_t$	$W_s + \mathbf{n} \cdot A_s$
USL-1F1B	$W_c + d \cdot A_h + A_t$	$W_s + d \cdot A_s$
ULHP (Ours)	$W_c + \mathbf{n} \cdot A_h + A_t$	$W_s + \mathbf{n} \cdot A_s$
SplitMind (Ours)	$\approx 0.5W_c + O(1) \cdot A_t$	$W_s + O(1) \cdot A_s$

3.3 USL-Aware Automated Parameter Tuning

Global configuration search for automated parameter tuning. Optimal pipeline scheduling and offloading are insufficient if key parameters, such as the split point or batch size, are improperly configured, potentially leading to pipeline bubbles or memory overflow. Consequently, automated parameter tuning is essential for practical USL deployment. Current approaches, relying on costly trial-and-error or error-prone symbolic derivation (§2.3), are inadequate. A superior alternative is the *profile-and-simulate* workflow, established in prior pipeline configuration research [17, 39, 48, 56, 67]. Rather than exhaustively evaluating candidates via full training, a profiler measures reusable base costs from a few real executions, outperforming symbolic derivation in accuracy. A simulator then extrapolates these measurements to estimate the execution time and memory usage of candidate configurations. Adapting this workflow to USL is non-trivial. Profiling must capture USL's unique constraints: client-fixed

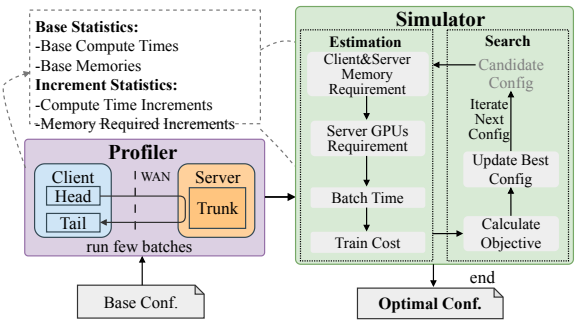


Figure 6. Overview of the adaptive configuration workflow.

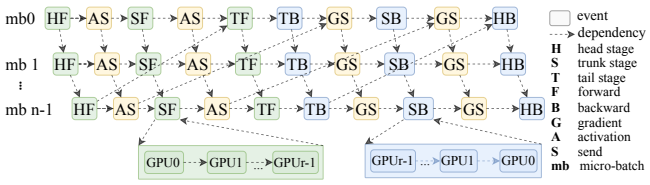


Figure 7. DAG of a training batch execution in USL.

head/tail stages, heterogeneous client-server GPUs, and hybrid communication paths spanning local and WAN links. Simulation also needs a redesign: existing models assume one-way dependencies, whereas a USL simulator must identify the U-shaped execution graph, whose critical path involves forward/backward computation, communication, and offloading, for time and memory accurate prediction. Furthermore, while prior methods optimize for throughput or memory, they largely ignore server rental cost, a primary constraint in resource-constrained USL deployments.

To overcome these challenges, we propose an *automated parameter tuning with USL-aware profile-and-simulate workflow.* We formulate parameter tuning as a constrained optimization problem over the feasible space of the split point and batch size (SP, B), aiming to minimize server rental cost and training latency while adhering to client memory constraints. As depicted in Figure 6, our method integrates USL-specific profiler and simulator. The profiler extracts reusable cost units from several real client-server runs. Using the reusable cost units, the simulator reconstructs USL-specific dependencies to evaluate candidates, estimating server memory demand to derive the required GPU count, estimating epoch training time, and calculating the total server rental cost (GPU count $\times$ duration time). Finally, it searches and selects the configuration with the lowest server rental cost.

USL-specific profiler. The profiler establishes a real USL execution path using one client and one server GPU. This preserves key runtime conditions of full USL training, including client-resident head/tail stages, the server trunk, the WAN client-server link, and the offloading path. To keep profiling lightweight, it executes only a few representative layers, leveraging the fact that transformer blocks in the same model exhibit similar computation and memory behavior [18]. This allows extrapolating measured costs to full-model configurations without full execution, avoiding reliance on theoretical parameters and capturing real heterogeneous GPU and WAN interactions with negligible overhead. During the execution, profiler extracts reusable cost units. It first measures computation time and memory usage for these reusable cost units in the client head, server trunk, and client tail at a base configuration (SP_{base}, B_{base}), defaulting to $SP_{\text{base}} = 1$ and $B_{\text{base}} = 4$. It then measure the offset growth of computation time and memory usage on both the client and server, by increasing the split point and batch size by 1 from the base configuration. Since SplitMind uses bubble-bounded offloading, the profiler does not separately measure offloading latency; memory stats are collected under offloading, so increments reflect the post-offloading footprint. Finally, the profiler measures the effective bandwidth of the client-server path to estimate the transfer time of activations and gradients. Our profiling process requires few GPU resources in both client and server side, and finishes in a few iterations, which is negligible compared to the thousands of iterations needed for large-scale fine-tuning (refer to §4.5). Detailed profile variables and profiling progress are provided in Appendix B.

USL execution graph aware simulator. The simulator overcomes a key limitation of existing pipeline simulators, which assume a linear one-way path and cannot identify the critical path in USL. As shown in Figure 7, we represent a training batch execution as a directed acyclic graph (DAG) of discrete events. Each event is an atomic operation (e.g., head/trunk computation, activation/gradient transfer), and edges denote execution dependencies. The forward path flows from the client head to the server trunk and back to the client tail; the backward path reverses this via gradient transfers. Given a candidate configuration (SP, B), the simulator instantiates this execution graph for all micro-batches, assigning event durations from profiled cost units. It then executes a discrete-event simulation over the execution graph, where an event is eligible to execute only after all antecedent events conclude and the requisite resource is idle. The simulator schedules ready events on shared resources (client GPU, server GPU, WAN link), and advances the simulation clock to the completion time of the next event. This event-driven approach avoids the inefficiencies of fixed-time-step simulation while preserving operation-level ordering. It captures cross-micro-batch dependencies and resource contention, determining whether WAN communication is hidden in bubbles or exposed on the critical path, enabling accurate batch time estimation. During replay, the simulator tracks resident model states, retained activations after offloading, and temporary transfer buffers to estimate client and server peak memory. Based on the estimated total server memory demand, it derives the minimum number of required server

Table 4. Model configurations used in experiments.

Model	Hidden Size	Layers	Vocabulary Size
Qwen3-4B	2560	36	151936
Qwen3-8B	4096	36	151936
LLaMA3-8B	4096	32	128256
Gemma2-9B	3584	42	256000
Qwen3-14B	5120	40	151936
Qwen3-32B	5120	64	151936

GPUs. Details of the training time and memory demand simulation are provided in Appendix C.

4 EXPERIMENTS

4.1 Experiment Setups

Testbed. We conduct experiments in a heterogeneous client-server setup mirroring a realistic USL deployment scenario. The client features an Intel Xeon Platinum 8352V CPU (2×36 cores), 256 GB of DRAM, and an NVIDIA GeForce RTX 4090 GPU (24 GB VRAM, PCIe 4.0 $\times$ 16). The server is equipped with two Intel Xeon Gold 6430 CPU (2×32 cores), 1TB of DRAM, and 8 $\times$ NVIDIA RTX A6000 GPU (48 GB VRAM, PCIe 4.0 $\times$ 16). The two machines are linked via a local area network and bandwidth is throttled to restrict interconnect throughput to 230 Mbps, as refer to [20].

Comparison baselines. Existing USL privacy-protection works rely on single-GPU emulation that ignores real network communication, failing to reflect real system performance. We therefore exclude them from comparison, as our system-level optimizations are orthogonal to these algorithmic enhancements. We compare our SplitMind against three USL training frameworks implemented in real system environment: (1) Naive-USL: standard USL framework without pipeline optimization. (2) USL-FthenB: F-then-B pipeline scheduling optimized USL framework. (3) USL-1F1B: 1F1B pipeline scheduling optimized USL framework. Details on these methods refer to §2.2. For a fair comparison, we implemented and tuned our SplitMind and all baselines using identical settings (PyTorch 2.6.0, CUDA 12.8). All pipeline-parallel training methods utilize *torch.distributed.pipelining* to facilitate USL, with NCCL [44] for intra-server GPU communication. We adopt LoRA-based fine-tuning [26, 32], which trains lightweight layerwise adapters while freezing the base model. Implementation details refer to Appendix A.

Models and datasets. We conduct experiments on six representative open-source large language models: LLaMA3-8B [59], Gemma2-9b [57], and four variants of the Qwen3 series (4B, 8B, 14B, and 32B) [64]. This diverse selection spans a broad spectrum of parameter scales (from 4B to 32B), allowing for a comprehensive evaluation of our framework's scalability and efficiency under varying computational budgets. The detailed model configurations, including the hidden size and the number of transformer layers, are summarized in Table 4. To ensure consistent evaluation across all models, we fix the sequence length to 512 and the micro-batch size to 1 for all experiments. We conduct fine-tuning experiments primarily on the UltraChat 200k dataset [12] to benchmark system efficiency and throughput.

4.2 Overall Performance

To comprehensively assess the performance of SplitMind, we evaluate the above six models on UltraChat-200k dataset for 5 epochs across three baselines (Naive-USL, USL-FthenB, and USL-1F1B) under varying batch sizes. Optimal split points for the baselines are determined manually through extensive experimental trials, whereas SplitMind (Adaptive) automatically optimizes both split points and batch sizes under system constraints. We further evaluate SplitMind (Fixed B), which optimizes only the split point while keeping the batch size fixed. Figure 8 summarizes the results across three metrics: *epoch training time* (top), *server memory allocated* (middle), and *server rental cost* (bottom). The x-axis in all plots represents the varying batch size. The baselines and SplitMind (Fixed B) are depicted as bar plots for specific batch sizes, with values in parentheses under each bar indicating the searched optimal configuration (split point, number of server GPUs) under each batch size setting. In contrast, SplitMind (Adaptive) is plotted as a horizontal dashed line for direct comparison, as it jointly searches for the globally optimal configuration of split points and batch sizes. Its chosen configuration is annotated in red parentheses as (split point, number of server GPUs, batch size). Additionally, OOM_C indicates client-side memory exhaustion even with minimal workload (i.e., only a single layer for head and tail segments), OOM_S denotes server failure with all eight GPUs even when client hosts the maximum possible layers, and $\text{OOM}_{C,S}$ signifies that both conditions are met, indicating hardware limits exceeded regardless of the split point.

Time cost evaluation. As shown in the top row of Figure 8, SplitMind generally achieves the lowest epoch training time across all evaluated models and batch sizes compared to the baselines. Compared with Naive-USL and USL-1F1B, SplitMind reduces average training time by 63.11% and 37.76% across all models, respectively. Compared with USL-FthenB, SplitMind (Fixed B) is also more efficient in most cases. While USL-FthenB may be faster in some cases, it uses more GPUs due to memory constraints. For example, with the Qwen3-4B model and a batch size of 32, USL-FthenB takes 24.32 hours per epoch with 4 A6000 GPUs on the server side, while SplitMind takes 28.00 hours with only a single A6000 GPU. Under the same server GPU allocation, SplitMind consistently achieves lower training time. This improvement comes from the U-shaped link-aware hybrid pipeline (§3.1), which reduces pipeline idle overhead by better overlapping computation and communication. Besides, SplitMind jointly optimizes the batch size and split point, yielding optimal and near-optimal results across most models. For Qwen3-32B,

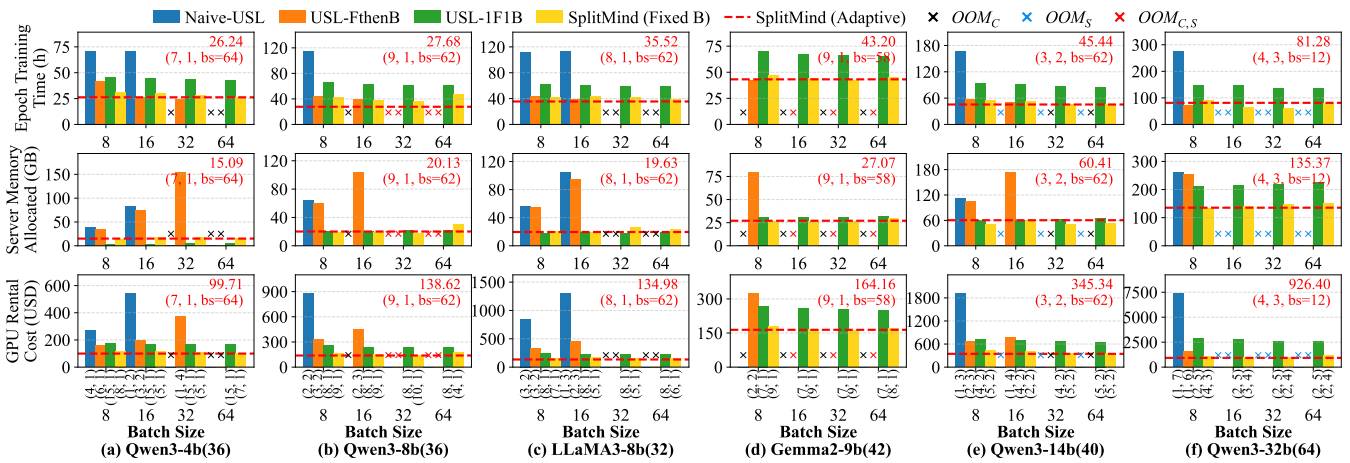


Figure 8. Comparison of epoch training time, server memory allocated, and server rental cost across models and batch sizes (Client: 1×RTX 4090, Server: 8×A6000).

SplitMind (Adaptive) prioritizes the configuration with the lowest server GPU requirement to minimize rental costs, even if it results in slightly higher time compared to SplitMind (Fixed B).

Server memory consumption evaluation. As shown in the middle row of Figure 8, As batch sizes increase, both Naive-USL and USL-FthenB frequently encounter out-of-memory (OOM) errors on either the client or server due to the accumulation of activations across batches or micro-batches. Although USL-1F1B helps alleviate memory concerns with its 1F1B scheduling, reducing memory usage and GPU requirements, SplitMind (Fixed B) consistently achieves the minimum server GPU allocation in all experimental configurations. This efficiency stems from our bubble-bounded inactive-tensor offloading strategy (see §3.2), which significantly alleviates server memory pressure without compromising performance. It is worth noting that although USL-1F1B may exhibit lower raw memory footprints than SplitMind (Fixed B) in some configurations, the number of allocated GPUs remains the same. Since resource rental is quantified by the discrete number of GPUs, SplitMind (Fixed B) maintains optimal cost-effectiveness. Moreover, SplitMind optimizes batch size and split points simultaneously, further reducing the number of required server-side GPUs.

Server rental cost. Server rental cost is the primary optimization objective of SplitMind because it directly reflects the monetary overhead of practical USL deployment, capturing both the number of allocated server GPUs and the training duration. As illustrated in the bottom row of Figure 8, compared with Naive-USL, both USL-FthenB and USL-1F1B significantly reduce rental costs by leveraging pipeline parallelism to accelerate training. Between the two, USL-1F1B is generally more cost-effective than USL-FthenB. Its 1F1B scheduling enables the immediate release of activations during the backward pass, effectively reducing peak server memory usage and, consequently, the number of required GPUs. However, exceptions exist. For instance, in specific cases like Qwen3-4B (batch size = 8), where both methods require the same number of GPUs, USL-FthenB proves slightly less expensive. This is because USL-FthenB incurs fewer pipeline bubbles in this configuration, resulting in a shorter training duration and lower total cost. In contrast, SplitMind achieves the lowest rental cost among all six models across all batch sizes. Most notably, for the massive Qwen3-32B model, SplitMind enables training on three server-side A6000 GPUs, achieving a 1.68× speedup and a 64.20% reduction in rental costs compared to the best competitor. This superior cost efficiency is structurally driven: SplitMind minimizes the hardware footprint (GPU count) via inactive-tensor offloading while reducing training time through hybrid pipeline parallelism. By jointly optimizing these two multiplicative factors of rental cost, our adaptive strategy successfully locates the global cost minimum that baselines fail to reach.

4.3 Performance under System Constraints

Impact of network bandwidth. To evaluate the impact of bandwidth, we use the Qwen3-8B and LLaMA3-8B models on the UltraChat-200k dataset with a batch size of 8. While Naive-USL, USL-FthenB, and USL-1F1B use the maximum feasible split point for this batch size, SplitMind searches for the optimal split point under this fixed constraint, resulting in a split point of 9. We vary inter-device bandwidth from 40 Mbps to 280 Mbps as shown in Figure 9. As bandwidth increases, all methods exhibit reduced training latency due to faster communication. Nevertheless, SplitMind demonstrates superior cost-effectiveness and resource efficiency across all settings. Compared to USL-FthenB, SplitMind achieves comparable (and slightly superior) latency while utilizing one

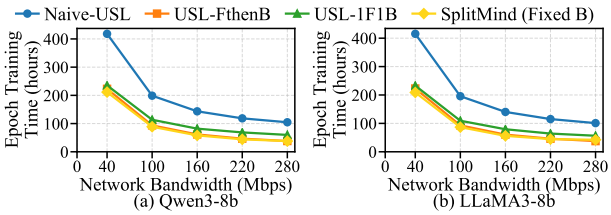


Figure 9. Comparison of epoch training time (hours) under different network bandwidth constraints.

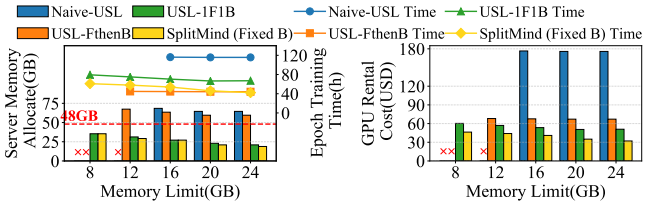


Figure 10. Comparison of server memory, epoch training time and GPU rental cost under different client memory limits. '×' denotes client-side OOM failures.

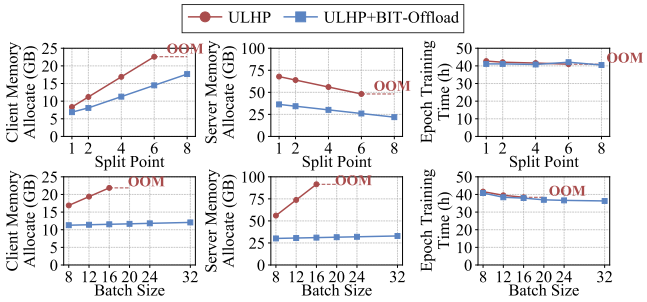


Figure 11. Impact of bubble-bounded inactive tensor offloading under different *SP* and *BS* settings.

fewer Server GPU. This reduction directly translates to significant savings in server rental overhead. Compared to USL-1F1B (which also utilizes a single Server GPU), SplitMind significantly accelerates the training process. For Qwen3-8B, it achieves speedups of 1.11×, 1.28×, 1.41×, 1.55×, and 1.57× at bandwidths of 40, 100, 160, 220, and 280 Mbps, respectively. The consistent performance advantage is attributed to our hybrid pipeline strategy. Unlike USL-FthenB, our method incorporates a 1F1B schedule during the tail phase. This design significantly mitigates the pipeline bubbles (idle time) typically seen in the final stages of execution, an optimization that proves effective across all bandwidth settings.

Impact of client memory capacity. We evaluate the impact of client-side memory constraints using the same experimental settings as above, including the model, baselines, and configuration search space. To simulate a diverse range of client device capabilities (from entry-level to mid-range), we vary the client memory limit from 8 GB to 24 GB in 4 GB increments. We measure server memory allocation, training time, and overall cost efficiency under these constraints, as shown in Figure 10. Experimental results demonstrate that SplitMind significantly reduces server GPU memory usage compared to Naive-USL and USL-FthenB across all tested conditions. Under strict client memory constraints (8 GB and 12 GB), both Naive-USL and USL-FthenB necessitate renting two Server GPUs to offload sufficient parameters, with Naive-USL even suffering from client-side OOM failures at the 8 GB limit. Compared to USL-1F1B, SplitMind maintains comparable training latency while identifying the optimal split point. Although SplitMind incurs marginally higher server VRAM usage than USL-1F1B, it remains well within the capacity of a single GPU, ensuring high resource utilization without incurring the cost of an additional device. Regarding overall rental costs, SplitMind demonstrates consistently superior cost efficiency across all client memory limits. When compared to USL-FthenB, although costs are comparable at moderate memory settings (16-20 GB), a significant gap emerges at 8, 12, and 24 GB. In these specific scenarios, USL-FthenB's excessive memory footprint necessitates the allocation of additional server GPUs, drastically inflating the hourly rental rate. Conversely, while USL-1F1B matches SplitMind in terms of server GPU quantity, its prolonged training duration inevitably results in higher total costs across all scenarios.

4.4 Ablation Study

To evaluate the efficacy of the bubble-bounded inactive-tensor offloading(BIT-Offload) mechanism in SplitMind, we conduct an ablation study using the Qwen3-8b model. We compare two configurations to quantify the performance gains:(1) the baseline *U-Shaped Link-Aware Hybrid Pipeline* (ULHP) **without offloading**; and(2) the full *SplitMind* framework with **BIT-Offload**.We analyze performance under two scenarios: varying the split point from 1 to 8 with a fixed batch size of 8, and varying the batch size from 8 to 32 with a fixed split point of 4. We report memory footprints and per-sample training latency in Figure 11. Results indicate that the baseline ULHP is strictly bound by memory constraints. Without offloading, client-side usage scales linearly, triggering out-of-memory failures when SP reaches 8 (at $B = 8$) or when B exceeds 16 (at $SP = 4$). In contrast, enabling offloading significantly alleviates memory pressure on both ends. For instance, at $B = 8$ and $SP = 6$, offloading reduces client memory usage from 22.59 GB to 14.48 GB and server usage from 48.10 GB to 25.98 GB. Crucially, across various split points and batch sizes, our method exhibits nearly overlapping training time trajectories compared to the baseline, demonstrating that the offloading time cost is effectively masked by the overlapping of computation and communication.

Table 5. Preprocessing overhead (time and rental cost).

Strategy	Profile Time (h)	Epoch Training Time (h)	Profile Rental Cost (USD)	Training Cost per Epoch (USD)
Qwen3-4B	0.073	26.24	0.056	19.94
Qwen3-8B	0.098	36.48	0.074	27.72
LLaMA3-8B	0.094	35.52	0.071	27.00
Gemma2-9B	0.095	43.20	0.072	32.83
Qwen3-14B	0.123	45.44	0.093	69.07
Qwen3-32B	0.101	81.28	0.077	185.32

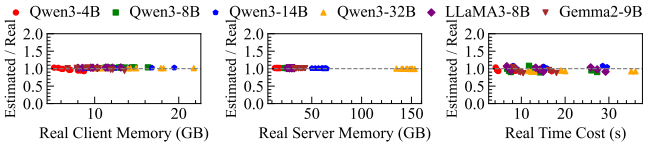


Figure 12. Estimation accuracy of client and server memory and training time across six models.

4.5 Preprocessing Overhead

To assess the overhead of SplitMind's preprocessing phase, defined as the profiling process for determining the optimal configuration, we measured the profile time and the full epoch training time across six models and translated these durations into exact financial metrics including profile rental cost and epoch training cost, as summarized in Table 5. Compared to the tens of hours required for training, profiling is minimal, consuming only 0.073–0.121 hours (<0.3% of total time). Financially, this process accounts for merely 0.04%–0.28% of the total rental expenditure. This demonstrates that the overhead for establishing the optimal setup is negligible compared to the resulting efficiency gains. This minimal overhead stems from our lightweight profiling strategy, which evaluates only a few representative transformer blocks under a minimal base configuration and infers performance across all other settings through analytical extrapolation.

4.6 Estimation Errors

To evaluate the accuracy of the SplitMind estimation framework, we compared our simulated estimates against actual performance measurements across six models. The estimation accuracy is quantified as the ratio of the simulated values to the actual system measurements, where a ratio of 1.0 indicates a perfect match, as depicted in Figure 12 The simulated results tightly cluster around the 1.0 baseline. Specifically, the average prediction errors (deviations from 1.0) for client memory and server memory are merely 2.67% and 1.25%, respectively. Concurrently, the average estimation error for the training latency remains remarkably low at 7.60%. This high level of robustness and low error rate across diverse models and configurations is primarily attributed to our lightweight profiling strategy and discrete-event simulation algorithm.

5 Related Work

System optimization for LLM training. System strategies are crucial for efficiently training LLMs at scale [30, 51, 54]. Pipeline parallelism, distributing model layers across devices for simultaneous execution, optimizes scheduling and boosts throughput [17, 28, 39, 41, 67, 70]. Memory optimizations like activation recomputation [8, 29] and tensor offloading [9, 31, 50, 52, 55, 71] address GPU capacity limits by either trading computation for space or utilizing external storage (such as CPU or NVMe) for parameter and activation hosting. While these cluster-scale techniques assume high-speed interconnects, SplitMind redesigns pipeline scheduling and memory offloading specifically for this bandwidth-constrained, U-shaped architecture.

System optimization for SL. Recent research has focused on improving the efficiency of SL. One approach optimizes split-point selection to minimize latency and energy consumption [53]. Communication compression techniques [2, 4, 69] and auxiliary networks for decoupled local updates [22, 38] have been proposed to alleviate bandwidth bottlenecks. Unlike these methods that often compromise final model accuracy due to lossy data compression and gradient approximations, SplitMind directly enhances training efficiency and reduces server rental costs while strictly preserving the integrity of the complete end-to-end training pipeline.

Split federated learning (SFL). SFL combines federated learning's parallel training with the resource efficiency of SL [58]. By enabling parallel forward and backward propagation across clients followed by global aggregation, SFL mitigates the computational bottlenecks and privacy concerns. Recent research integrates PEFT methods to reduce communication overhead [32, 33], and proposes scheduling strategies to optimize server-side concurrency for high-throughput training [20, 23]. Furthermore, some studies leverages pipelining among multi-client to overlap computation with communication, reducing idle time and boosting throughput [36, 66]. Rather than scaling across multiple clients, our work targets a single-client and single-server topology. By resolving the U-shaped architecture's memory and pipeline inefficiencies, we push the limits of LLM training on individual memory-constrained devices.

Profile-and-simulate based configuration. To avoid the prohibitive cost of exhaustive search, recent systems widely adopt a profile-and-simulate paradigm. PipeDream[39] and DAPPLE[17] profile computation and communication costs to evaluate partitioning and placement strategies under device and memory constraints. AdaPipe[56] extends this workflow to memory-saving policies by jointly evaluating scheduling and recomputation decisions. Recent schedule-oriented systems, such as ZeroBubble[48] and FlexPipe[67], further refine the simulation stage with fine-grained execution orders and schedule templates. These methods primarily target

throughput and memory optimization in tightly coupled linear pipelines. In contrast, SplitMind natively models the complex U-shaped critical paths and cross-device heterogeneity, explicitly optimizing for the overall rental cost rather than raw performance alone.

6 Conclusion

This paper introduces SplitMind, a novel U-shaped split learning (USL) framework that integrates a U-shaped link-aware hybrid pipeline, bubble-bounded inactive-tensor offloading, and USL-aware automated parameter tuning to facilitate the practical deployment of USL. Extensive evaluation demonstrates that SplitMind significantly enhances training efficiency, reduces server rental costs, and supports larger model scales and batch sizes compared to standard USL frameworks and their pipeline-optimized variants. By establishing a robust foundation for efficient and privacy-preserving distributed LLM training, this work enables more accessible and cost-effective customized AI services.

References

[1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 308–318.

[2] Ahmad Ayad, Melvin Renner, and Anke Schmeink. 2021. Improving the communication and computation efficiency of split learning for iot applications. In *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 01–06.

[3] Mislav Balunovic, Dimitar Dimitrov, Nikola Jovanović, and Martin Vechev. 2022. Lamp: Extracting text from gradients with language model priors. *Advances in Neural Information Processing Systems* 35 (2022), 7641–7654.

[4] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Riabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative Inference and Fine-tuning of Large Models. In *The 61st Annual Meeting Of The Association For Computational Linguistics*.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Ma teusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *ArXiv* abs/2005.14165 (2020). https://api.semanticscholar.org/CorpusID:218971783

[6] Linxiao Cao, Yifei Zhu, and Wei Gong. 2024. Sfprompt: Communication-efficient split federated fine-tuning for large pre-trained models over resource-limited devices. *arXiv preprint arXiv:2407.17533* (2024).

[7] Guanzhong Chen, Zhenghan Qin, Mingxin Yang, Yajie Zhou, Tao Fan, Tianyu Du, and Zenglin Xu. 2024. Unveiling the vulnerability of private fine-tuning in split-based frameworks for large language models: A bidirectionally enhanced attack. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2904–2918.

[8] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).

[9] Chien chin Huang, Gu Jin, and Jinyang Li. 2020. SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping. *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (2020). https://api.semanticscholar.org/CorpusID:211019602

[10] Jieren Deng, Yijue Wang, Ji Li, Chao Shang, Hang Liu, Sanguthevar Rajasekaran, and Caiwen Ding. 2021. Tag: Gradient attack on transformer-based language models. *arXiv preprint arXiv:2103.06819* (2021).

[11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *North American Chapter of the Association for Computational Linguistics*. https://api.semanticscholar.org/CorpusID:52967399

[12] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3029–3051.

[13] Minxin Du, Xiang Yue, Sherman SM Chow, and Huan Sun. 2023. Sanitizing sentence embeddings (and labels) for local differential privacy. In *Proceedings of the ACM Web Conference 2023*. 2349–2359.

[14] Minxin Du, Xiang Yue, Sherman SM Chow, Tianhao Wang, Chenyu Huang, and Huan Sun. 2023. Dp-forward: Fine-tuning and inference on language models with differential privacy in forward pass. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2665–2679.

[15] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*. Springer, 265–284.

[16] Ege Erdoğan, Alptekin Küpçü, and A Ercüment Çiçek. 2022. Unsplit: Data-oblivious model inversion, model stealing, and label inference attacks against split learning. In *Proceedings of the 21st Workshop on Privacy in the Electronic Society*. 115–124.

[17] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, Lansong Diao, Xiaoyong Liu, and Wei Lin. 2020. DAPPLE: a pipelined data parallel approach for training large models. *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (2020). https://api.semanticscholar.org/CorpusID:220302383

[18] Yangyang Feng, Minhui Xie, Zijie Tian, Shuo Wang, Youyou Lu, and Jiwu Shu. 2023. Mobius: Fine Tuning Large-Scale Models on Commodity GPU Servers. *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2023). https://api.semanticscholar.org/CorpusID:256391189

[19] Xinben Gao and Lan Zhang. 2023. {PCAT}: Functionality and data stealing from split learning by {Pseudo-Client} attack. In *32nd USENIX security symposium (USENIX Security 23)*. 5271–5288.

[20] Yunqi Gao, Bing Hu, Mahdi Boloursaz Mashhadi, Wei Wang, and Mehdi Bennis. 2025. PipeSFL: A Fine-Grained Parallelization Framework for Split Federated Learning on Heterogeneous Clients. *IEEE Transactions on Mobile Computing* 24 (2025), 1774–1791. https://api.semanticscholar.org/CorpusID:273767335

[21] Otkrist Gupta and Ramesh Raskar. 2018. Distributed learning of deep neural network over multiple agents. *Journal of Network and Computer Applications* 116 (2018), 1–8.

[22] Dong-Jun Han, Hasnain Irshad Bhatti, Jungmoon Lee, and Jaekyun Moon. 2021. Accelerating federated learning with split learning on locally generated losses. In *ICML 2021 workshop on federated learning for user privacy and data confidentiality. ICML Board*.

[23] Qiang He, Kaibin Wang, Zeqian Dong, Liang Yuan, Feifei Chen, Hai Jin, and Yun Yang. 2025. Hourglass: Enabling Efficient Split Federated Learning with Data Parallelism. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1317–1333.

[24] Zecheng He, Tianwei Zhang, and Ruby B Lee. 2019. Model inversion attacks against collaborative inference. In *Proceedings of the 35th annual computer security applications conference*. 148–162.

[25] Shlomo Hoory, Amir Feder, Avichai Tendler, Sofia Erell, Alon Peled-Cohen, Itay Laish, Hootan Nakhost, Uri Stemmer, Ayelet Benjamini, Avinatan Hassidim, et al. 2021. Learning and evaluating a differentially private pre-trained language model. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 1178–1189.

[26] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[27] Haichen Huang, Jiarui Fang, Hongxin Liu, Shenggui Li, and Yang You. 2022. Elixir: Train a Large Language Model on a Small GPU Cluster. *ArXiv* abs/2212.05339 (2022). https://api.semanticscholar.org/CorpusID:254564302

[28] Yanping Huang, Yonglong Cheng, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, and Z. Chen. 2018. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Neural Information Processing Systems*. https://api.semanticscholar.org/CorpusID:53670168

[29] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence C. McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing Activation Recomputation in Large Transformer Models. *ArXiv* abs/2205.05198 (2022). https://api.semanticscholar.org/CorpusID:248693351

[30] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704* (2020).

[31] Changyue Liao, Mo Sun, Zihan Yang, Jun Xie, Kaiqi Chen, Binhang Yuan, Fei Wu, and Zeke Wang. 2025. Ratel: Optimizing Holistic Data Movement to Fine-tune 100B Model on a Consumer GPU. *2025 IEEE 41st International Conference on Data Engineering (ICDE)* (2025), 292–306. https://api.semanticscholar.org/CorpusID:280697286

[32] Zheng Lin, Xuanjie Hu, Yuxin Zhang, Zhe Chen, Zihan Fang, Xianhao Chen, Ang Li, Praneeth Vepakomma, and Yue Gao. 2024. Splitlora: A split parameter-efficient fine-tuning framework for large language models. *arXiv preprint arXiv:2407.00952* (2024).

[33] Zheng Lin, Yuxin Zhang, Zhe Chen, Zihan Fang, Xianhao Chen, Praneeth Vepakomma, Wei Ni, Jun Luo, and Yue Gao. 2025. HSplitLoRA: A Heterogeneous Split Parameter-Efficient Fine-Tuning Framework for Large Language Models. *arXiv preprint arXiv:2505.02795* (2025).

[34] Zihan Liu, Yizhen Wang, Rui Wang, and Sai Wu. 2025. DualGuard: A Parameter Space Transformation Approach for Bidirectional Defense in Split-Based LLM Fine-Tuning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 17065–17080.

[35] Song Lyu, Zheng Lin, Guanqiao Qu, Xianhao Chen, Xiaoxia Huang, and Pan Li. 2023. Optimal resource allocation for u-shaped parallel split learning. In *2023 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 197–202.

[36] Jian Ma, Xinchen Lyu, Jun Jiang, Qimei Cui, Haipeng Yao, and Xiaofeng Tao. 2025. SplitFrozen: Split Learning with Device-side Model Frozen for Fine-Tuning LLM on Heterogeneous Resource-Constrained Devices. *arXiv preprint arXiv:2503.18986* (2025).

[37] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. 2024. Split-and-denoise: protect large language model inference with local differential privacy. In *Proceedings of the 41st International Conference on Machine Learning*. 34281–34302.

[38] Srijith Nair, Michael Lin, Peizhong Ju, Amirreza Talebi, Elizabeth Serena Bentley, and Jia Liu. 2025. FSL-SAGE: Accelerating Federated Split Learning via Smashed Activation Gradient Estimation. In *Forty-second International Conference on Machine Learning*.

[39] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei A. Zaharia. 2019. PipeDream: generalized pipeline parallelism for DNN training. *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (2019). https://api.semanticscholar.org/CorpusID:202488191

[40] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-efficient pipeline-parallel dnn training. In *International Conference on Machine Learning*. PMLR, 7937–7947.

[41] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–15.

[42] Khoa Nguyen, Tanveer Khan, and Antonis Michalas. 2023. Split without a leak: Reducing privacy leakage in split learning. In *International Conference on Security and Privacy in Communication Systems*. Springer, 321–344.

[43] NVIDIA. 2022. NVIDIA Announces Hopper Architecture, the Next Generation of Accelerated Computing. https://nvidianews.nvidia.com/news/nvidia-announces-hopper-architecture-the-next-generation-of-accelerated-computing. States that every GPU in DGX H100 systems is connected by fourth-generation NVLink, providing 900 GB/s connectivity. Accessed 2026-05-12.

[44] NVIDIA. 2026. NVIDIA Collective Communications Library (NCCL). https://developer.nvidia.com/nccl. Official NVIDIA developer page. NCCL implements multi-GPU and multi-node communication primitives optimized for NVIDIA GPUs and networking. Accessed 2026-05-12.

[45] Jay H Park, Gyeongchan Yun, Chang M Yi, Nguyen T Nguyen, Seungmin Lee, Jaesik Choi, Sam H Noh, and Choi Young-ri. 2020. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 307–321.

[46] Dario Pasquini, Giuseppe Ateniese, and Massimo Bernaschi. 2021. Unleashing the tiger: Inference attacks on split learning. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. 2113–2129.

[47] Maarten G Poirot, Praneeth Vepakomma, Ken Chang, Jayashree Kalpathy-Cramer, Rajiv Gupta, and Ramesh Raskar. 2019. Split learning for collaborative deep learning in healthcare. *arXiv preprint arXiv:1912.12115* (2019).

[48] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. 2023. Zero Bubble Pipeline Parallelism. *ArXiv* abs/2401.10241 (2023). https://api.semanticscholar.org/CorpusID:267060979

[49] Xinchi Qiu, Ilias Leontiadis, Luca Melis, Alex Sablayrolles, and Pierre Stock. 2023. Evaluating privacy leakage in split learning. *arXiv preprint arXiv:2305.12997* (2023).

[50] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep learning. *SC21: International Conference for High Performance Computing, Networking, Storage and Analysis* (2021), 1–15. https://api.semanticscholar.org/CorpusID:233289729

[51] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2020). https://api.semanticscholar.org/CorpusID:221191193

[52] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 551–564.

[53] Eric Samikwa, Antonio Di Maio, and Torsten Braun. 2022. Ares: Adaptive resource-aware split learning for internet of things. *Computer Networks* 218 (2022), 109380.

[54] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[55] Xiaoyang Sun, Wen Wang, Shenghao Qiu, Renyu Yang, Songfang Huang, Jie Xu, and Zheng Wang. 2022. STRONGHOLD: Fast and Affordable Billion-Scale Deep Learning Model Training. *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis* (2022), 1–17. https://api.semanticscholar.org/CorpusID:257158304

[56] Zhenbo Sun, Huanqi Cao, Yuanwei Wang, Guanyu Feng, Shengqi Chen, Haojie Wang, and Wenguang Chen. 2024. AdaPipe: Optimizing Pipeline Parallelism with Adaptive Recomputation and Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 86–100. doi:10.1145/3620666.3651359

[57] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118* (2024).

[58] Chandra Thapa, Pathum Chamikara Mahawaga Arachchige, Seyit Camtepe, and Lichao Sun. 2022. Splitfed: When federated learning meets split learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 8485–8493.

[59] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aur'elien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *ArXiv* abs/2302.13971 (2023). https://api.semanticscholar.org/CorpusID:257219404

[60] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. 2018. Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564* (2018).

[61] Teng Wang, Lindong Zhai, Tengfei Yang, Zhucheng Luo, and Shuanggen Liu. 2024. Selective privacy-preserving framework for large language models fine-tuning. *Information Sciences* 678 (2024), 121000.

[62] Maoqiang Wu, Guoliang Cheng, Dongdong Ye, Jiawen Kang, Rong Yu, Yuan Wu, and Miao Pan. 2023. Federated split learning with data and label privacy preservation in vehicular networks. *IEEE Transactions on Vehicular Technology* 73, 1 (2023), 1223–1238.

[63] Biwei Yan, Kun Li, Minghui Xu, Yueyan Dong, Yue Zhang, Zhaochun Ren, and Xiuzhen Cheng. 2024. On protecting the data privacy of large language models (llms): A survey. *arXiv preprint arXiv:2403.05156* (2024).

[64] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxin Yang, Jingren Zhou, Jingren Zhou, Junyan Lin, Kai Dang, Keqin Bao, Ke-Pei Yang, Le Yu, Li-Chun Deng, Mei Li, Min Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shi-Qiang Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yi-Chao Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *ArXiv* abs/2505.09388 (2025). https://api.semanticscholar.org/CorpusID:278602855

[65] Songge Zhang, Guoliang Cheng, Zuguang Li, and Wen Wu. 2024. SplitLLM: Hierarchical Split Learning for Large Language Model over Wireless Network. *2024 IEEE Globecom Workshops (GC Wkshps)* (2024), 1–6. https://api.semanticscholar.org/CorpusID:275820368

[66] Zihan Zhang, Philip Rodgers, Peter Kilpatrick, Ivor Spence, and Blesson Varghese. 2024. PiPar: Pipeline parallelism for collaborative machine learning. *J. Parallel and Distrib. Comput.* 193 (2024), 104947.

[67] Hairui Zhao, Qi Tian, Hongliang Li, and Zizhong Chen. 2025. FlexPipe: maximizing training efficiency for transformer-based models with variable-length inputs. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) *(USENIX ATC '25)*. USENIX Association, Article 9, 17 pages.

[68] Kai Zhao and Zhaohui Yang. 2025. Efficient Federated Split Learning for Large Language Models over Communication Networks. *arXiv preprint arXiv:2504.14667* (2025).

[69] Fei Zheng, Chaochao Chen, Lingjuan Lyu, and Binhui Yao. 2023. Reducing communication for split learning by randomized top-k sparsification. *arXiv preprint arXiv:2305.18469* (2023).

[70] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.

[71] Quan Zhou, Haiquan Wang, Xiaoyan Yu, Cheng Li, Youhui Bai, Feng Yan, and Yinlong Xu. 2023. MPress: Democratizing Billion-Scale Model Training on Multi-GPU Servers via Memory-Saving Inter-Operator Parallelism. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 556–569. doi:10.1109/HPCA56546.2023.10071077

[72] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. *Advances in neural information processing systems* 32 (2019).